An Architectural Digital Twin for Pre-Deployment Verification and CI/CD Gating in Distributed Middleware Systems (Industry Track)

TODO(author)
TODO(affiliation) — Industrial Track requires at least one author with an industry affiliation
TODO(city), TODO(country)
TODO(email)

Abstract

Distributed systems built on publish/subscribe and broker-mediated middleware (DDS, ROS 2, enterprise message buses) are deployed across multi-core processing units, operator consoles, and networked execution nodes. Their most damaging defects are not local: overlapping CPU core allocations, incompatible Quality-of-Service (QoS) contracts between a publisher and its subscribers, topics that lost their last consumer during a refactor, and circular package dependencies all pass unit and integration testing and surface as field failures. Reproducing them requires the assembled target hardware, which is exactly the resource a continuous-delivery pipeline cannot hold.

This paper reports on **System as a Graph (SaaG)**, an architectural digital twin for pre-deployment verification and CI/CD gating, built for a mission-critical pub/sub program. The twin is *static*: reconstructed fresh from the candidate system version each pipeline run, not continuously live-synchronized — the dynamic half (telemetry overlay, drift detection) is specified but not yet built, and we say so explicitly. We contribute three things: (1) the **digital twin model and its requirements baseline** — five component classes, six structural relation types, and six typed projection rules deriving logical dependencies from physical pub/sub linkage, backed by 112 verifiable requirements developed with the program's engineering organization; (2) a **CI/CD gate** — an implemented severity-based exit-code gate, paired with a scored suitability model specified but not yet built, over a checks catalog covering orphaned topics, dependency cycles, and cascade-impact simulation; and (3) **deployment experience** — measurements from the program's pipeline and an honest account of the distance between specification and prototype. We evaluate the prototype primarily on a family of five systems in the deployment's own domain — air traffic management — spanning 29 to 444 components with QoS mix, fan-out shape, and criticality proportion held fixed across scale, isolating scale as the sole independent variable: verification cost grows gently (0.01 s to 1.01 s end-to-end) but the detector catalog's agreement with a cascade oracle declines monotonically from slight to chance as the system grows (Cohen's κ: 0.12 at 29 components to −0.05 at 444). A secondary check across six other synthetic domains plus three transcribed real architectures finds the same near-chance pattern on other large synthetic graphs, but materially better agreement on the transcribed ones — a small-sample ($n = 3$) split left unresolved between scale and synthetic generation as the cause. We argue that the gap between what such a system is specified to check and what any implementation actually checks — and that verification quality is not scale-invariant even within one domain — are the most useful things an industrial report can make explicit.

CCS Concepts

• **Software and its engineering** → **Software verification and validation**; *Software configuration management and version control systems*; • **Networks** → *Network reliability*.

Keywords

architectural digital twin, pre-deployment verification, pub/sub middleware, QoS contracts, CI/CD gating, DDS

ACM Reference Format:
TODO(author). 2026. An Architectural Digital Twin for Pre-Deployment Verification and CI/CD Gating in Distributed Middleware Systems (Industry Track). In *Proceedings of 27th ACM/IFIP International Middleware Conference (Industrial Track) (Middleware '26).* ACM, New York, NY, USA, 7 pages.

1 Introduction

Continuous delivery has made building, testing, and shipping software artifacts routine. It has not made *deploying them into an assembled distributed system* routine. In programs governed by pub/sub middleware — Data Distribution Service (DDS), ROS 2, enterprise message buses — the scope of what a pipeline can test remains confined to a software unit's own logic and its immediate interfaces.

1.1 The Pre-Deployment Verification Gap

In a large distributed pub/sub deployment, applications run on multi-core processing nodes, operator consoles, and networked execution units. Introducing one updated software unit creates non-local interactions that no unit test observes:

- **Core contention.** Overlapping core pinning, or allocations exceeding a node's physical core count. The build is green; the timing violation appears only under field load.
- **QoS contract incompatibility.** A subscriber requests `TRANSIENT_LOCAL` durability from a `VOLATILE` publisher, or `RELIABLE` delivery from a `BEST_EFFORT` writer. In DDS these are not errors — endpoints simply never match, and the subscriber receives nothing.
- **Silent topic disconnection.** A refactor removes a topic's last subscriber, or two units bind the same topic name to divergent payload definitions. Both compile.
- **Circular package dependency.** A dependency edge added for convenience closes a cycle that only manifests as an initialization deadlock on the target.

Table 1: The two artifacts this paper reports on. Measurements from one are never presented as evidence for the other.

	SaaG-D (deployed)	SaaG-P (prototype)
What it is	Capability specified by the 112 requirements, integrated into the delivery pipeline	Open implementation of the graph model and a subset of the checks
Data sources	CMDB, source repos, package repo, network topology descriptors	Descriptor files (JSON), a scenario generator, transcribed open-source architectures
Evaluation	§5.1, [I]	§5.2, [P]

Standing up the full target hardware for every candidate build is prohibitively expensive, so teams fall back on partial staging environments — which is precisely where non-local defects hide. The result is a class of defects that is cheap to detect statically and expensive to detect any other way.

1.2 What This Paper Reports

We describe SaaG, an approach that models the system as a typed graph and audits it before installation. The paper is deliberately structured around three artifacts at different maturity levels (§2–§3.4 the twin and its requirements, §4 the gate's implemented and specified halves, §5 deployment and prototype measurements), and is explicit throughout about which is which.

Our central observation is the one we would have wanted before starting: **the expensive part is not building the graph, it is the semantic depth of the checks.** Graph construction and structural audits are cheap and were finished early. The checks that catch the defects in §1.1 — QoS contract matching with request/offer semantics, core-allocation conformance, design-versus-runtime drift — require domain models that the graph alone does not supply. §3.4 states exactly which of these are specified and not yet built, and why.

2 System Overview and Graph Model

2.1 Two Artifacts, One Name

Distinguishing these matters for reading §5, so we state it once, plainly:

SaaG-D is program-internal; SaaG-P's replication package is at TODO(url). Where SaaG-D implements a check that SaaG-P does not, §3.4 says so.

What kind of digital twin this is. "Digital twin" is used precisely, not loosely: SaaG-P builds a *static* twin, reconstructed fresh for each candidate build (§2.4), not a model kept continuously synchronized with the running system. A full digital twin's dynamic half — overlaying live telemetry and detecting drift against the design — is specified but not implemented; §3.4 lists it alongside the other specified-but-unbuilt checks rather than folding it silently into what the prototype already does. Everything this paper reports as a capability of SaaG-P is the static half.

2.2 Model Setup Data Generation

The specification requires model setup data to be produced in a controlled, traceable way from four external sources: the CMDB (project, platform, and system version metadata, with the effective version marked), the source and installation-script repositories, the software package repository, and the network topology descriptor. Each dataset is tagged with project, platform, and system version, and every mandatory-field or schema failure is recorded against its source.

The candidate-evaluation path matters most for gating: the software unit version inventory is rebuilt by substituting the *candidate* version of one unit into the otherwise-unchanged inventory, so the audited graph is the system as it would be after installation.

In SaaG-P, repository-side extraction is implemented — cloning, XML/IDL-adjacent topic parsing, and per-unit code metrics — but configuration-management and network-topology ingestion are not; those inputs are supplied as descriptor files. §3.4 records this.

2.3 Graph Model

SaaG represents the system as a typed, weighted, directed multigraph $G = (V, E, \tau_V, \tau_E, w)$ over five component classes: **applications** (deployable software units), **libraries** (shared modules), **topics** (pub/sub channels carrying QoS contracts), **brokers** (middleware routers and DDS participants), and **nodes** (processing units and operator consoles). Figure 1 shows how these are assembled and audited.

Six relation types are imported directly from the descriptors: `PUBLISHES_TO`, `SUBSCRIBES_TO` (component ↔ topic), `ROUTES` (broker → topic), `RUNS_ON` (component → node), `CONNECTS_TO` (node ↔ node), and `USES` (component → library).

Derived dependencies. Physical linkage is not dependency. A subscriber depends on the publishers of the topics it consumes, but the message flows the other way, so SaaG derives a `DEPENDS_ON` relation by six typed projection rules, oriented *from dependent to dependency* — against the direction of data flow — covering application-to-application coupling through shared topics (including transitive library chains), application-to-broker coupling, the lifting of both to node level, library usage, and broker colocation. Where two components share several topics, one edge carries the maximum topic weight and the shared-topic count as a coupling strength. Worth stating plainly, since pub/sub diagrams conventionally draw arrows as data flow: **data flows publisher → subscriber; dependency points subscriber → publisher.**

QoS-derived weights. Each topic carries an intrinsic weight combining its QoS contract with payload size; component weights propagate upward from the topics they touch, with library weights amplified by consumer fan-out to reflect blast radius. The weighting scheme and its calibration are the subject of a companion manuscript [9]; for this paper it is enough that edges and components carry a criticality scalar in $[0, 1]$ used to rank findings.

2.4 Candidate Isolation

Concurrent pipeline runs must not corrupt one another or the baseline. Each candidate evaluation constructs a process-specific model from the candidate unit plus the target system version's other units, under an independent operation identifier, and analysis never mutates the baseline model.

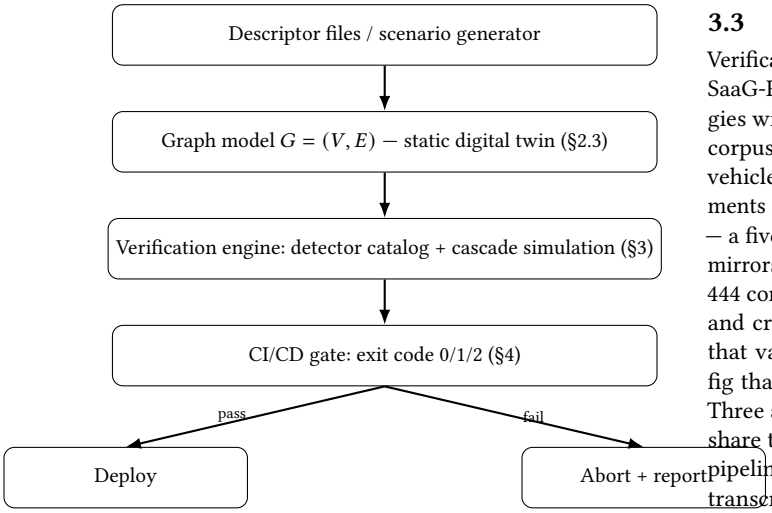


Figure 1: SaaG pipeline overview. The graph (SaaG-P's static twin) is rebuilt per candidate build (§2.4) and audited before the gate decision; SaaG-D specifies a scored variant of the same gate (§4.2) not yet built in the prototype.

3 Verification Engine

3.1 What the Prototype Checks Today

SaaG-P implements a catalog of structural and criticality-weighted detectors over the derived dependency graph. Those bearing directly on the defect classes of §1.1:

- **Orphaned topics** — publishers with no subscribers or vice versa, the structural signature of the refactor in §1.1.
- **Dependency cycles** — strongly connected components over the application dependency subgraph.
- **SPOFs and bottlenecks** — articulation points and bridge edges, ranked by propagated QoS weight.
- **Structural concentration** — broker overload, topic fan-out, hub-and-spoke concentration, deep pipelines, god components.

Detectors emit findings with a severity (CRITICAL/HIGH/MEDIUM), affected components, the rule that fired, and a recommendation.

3.2 Cascade Impact Simulation

Beyond static rules, SaaG-P simulates component failure to estimate blast radius before deployment. Failure is injected at a component and propagated along structural pub/sub edges — never along the derived `DEPENDS_ON` edges, an independence property enforced by the prototype's test suite so that simulation cannot silently consume its own derived output. The result ranks components by cascade impact, which is what §5.2 uses as a reference oracle when scoring the detector catalog.

3.3 Scenario Generation

Verification requires system models that do not yet exist in the field. SaaG-P includes a scenario generator producing synthetic topologies with controlled scale, QoS mix, and clustering, plus a curated corpus of fifteen committed scenarios: ten spanning autonomous vehicles, IoT, financial trading, healthcare, and enterprise deployments at various scales, plus — the evaluation's primary focus, §5.2 — a five-scenario family in the air-traffic-management domain that mirrors the deployment motivating this paper (§2.2), spanning 29 to 444 components and sharing an identical QoS-mix, fan-out-shape, and criticality-proportion profile so that scale is the only thing that varies between them. Each is hash-pinned against the config that generates it, so published numbers remain reproducible. Three architectures transcribed from open-source systems (§5.2.3) share the same descriptor schema and load through the identical pipeline, letting the secondary evaluation compare generated and transcribed topologies directly rather than describing the latter only qualitatively.

3.4 Specified but Not Yet Implemented

The following are required of SaaG-D and are **not** implemented in SaaG-P. We list them because the gap is the most transferable finding in this paper: each is a check whose value is obvious and whose cost is dominated by acquiring a domain model the graph does not carry.

Note what the table does *not* say: none of these are blocked on graph technology. Four of the seven are blocked on **input acquisition** — the information exists in the program but not in a form the model ingests. That ratio is our main lesson for teams attempting the same thing.

4 CI/CD Gating

4.1 The Implemented Gate

SaaG-P integrates as a command-line invocation. It constructs the candidate graph, runs the detectors, optionally writes the full finding set as JSON for downstream tooling, and encodes its decision in the process exit status: **2** if any CRITICAL or HIGH finding is present, **1** if any finding is present, **0** if none. Standard CI runners treat non-zero as failure, so a two-line pipeline stage gets absolute severity gating with no plugin (Figure 2).

Two limitations are worth stating because they are the difference between a usable gate and a tolerated one:

- **The gate is absolute, not delta-aware.** It scores the candidate graph on its own, not against the merge base, so a codebase carrying pre-existing HIGH findings fails every build until cleared — in practice, teams disable the gate. Not hypothetical here: at CRITICAL/HIGH severity, **all sixteen systems we evaluated** (five ATM scale points, §5.2.1–§5.2.2; eleven secondary-corpus scenarios, §5.2.3) **return exit code 2 at every oracle seed tested** — exactly the failure mode that gets a gate disabled rather than heeded. Delta semantics and a waiver register are the first changes we would make; a learned, delta-aware, simulation-verified gate is the companion manuscript's [9] contribution, cited here as the direction rather than duplicated.

Table 2: Specified capabilities not yet implemented in SaaG-P. None are blocked on graph technology; four of the seven are blocked on input acquisition (§3.4).

Capability	Why it is not yet built
QoS contract conformance — request/offer matching of durability, reliability, transport priority per topic	Flags a criticality-gap *anomaly*, not a contract violation: needs per-endpoint QoS at reader/writer granularity; the model currently attaches QoS to the topic.
Payload schema consistency — same topic name, divergent content definition	Requires parsing/comparing IDL/message definitions across units; the extractor does not yet resolve type definitions.
Core allocation conformance — capacity, conflicting pinning, dedicated high-performance cores	Core/memory sizes are node attributes but no rule evaluates them; needs core-assignment descriptors outside the prototype's inputs.
OS and runtime configuration audit — OS settings vs. allocation	Same cause: not in the prototype's input set.
Architectural drift — designed vs. observed topology (the dynamic half of the digital twin, §2.1)	Needs a field-record store and a graph-diff over observed edges; neither exists in the prototype.
Installation suitability scoring — four headings, per-rule weights, blocking flags, aggregate score	The gate is severity-based, not score-based (§4); specified in §4.2 as a requirement, not a result.
CMDB and network topology ingestion	Prototype consumes descriptor files instead.

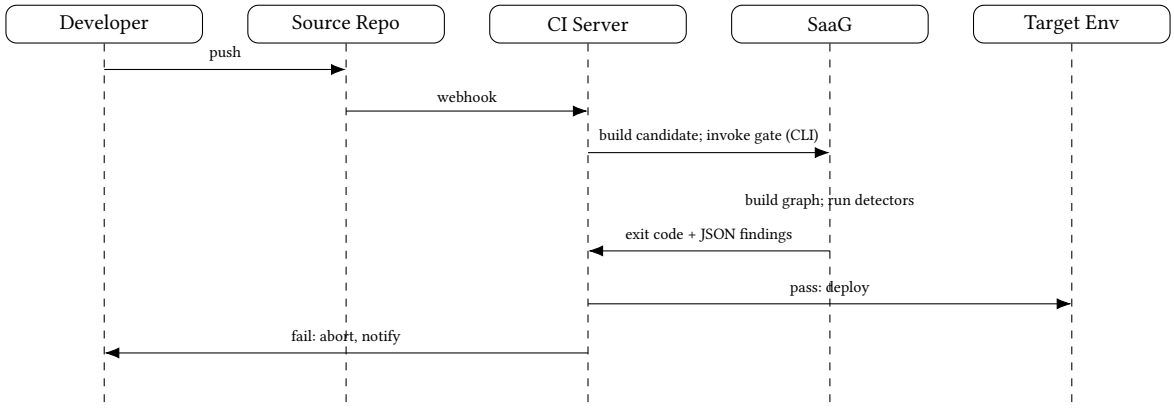


Figure 2: Gate integration sequence. The prototype's decision is the process exit code (§4.1).

- **There is no severity budget.** One HIGH finding and three hundred are the same decision.

4.2 The Specified Scoring Model

SaaG-D is specified to replace the exit-code gate with a scored evaluation over four headings — structural/architectural, interface/topic/communication, dependency/integration, and resource/performance conformance. Each rule carries an identifier, heading, severity, weight, acceptance criterion, and blocking flag. A finding at critical severity, or a blocking-rule violation, yields a non-conforming decision **independently of the aggregate score** — the score is for reporting and trend analysis; blocking is categorical. Multi-unit evaluations run under independent operation identifiers, returning per-unit decisions plus an aggregate result in machine-processable form.

We report this as specification, not as measurement: it is not implemented in SaaG-P, and §5 makes no claim about its behaviour.

4.3 Findings and Reports

Each finding carries an identifier, type, description, affected entity, the rule or acceptance criterion it derives from, supporting evidence, and a severity level. This evidence-bearing structure is what makes a failing gate actionable rather than merely obstructive: the developer receives the offending component and the rule, not a build log.

5 Evaluation

Two evaluations, kept separate. §5.1 reports the deployment ([I]); §5.2 reports the open prototype ([P]) and is fully reproducible. Neither substitutes for the other: the deployment numbers describe real engineering impact but cannot be independently verified; the prototype numbers can be rerun by any reader but describe a subset of the capability.

5.1 -

– TODO(data)]Deployment Experience [I] — TODO(data)

Status: awaiting cleared data. Every figure below must carry an n, a window, and provenance, and appear only where the corresponding check actually ran in the deployment; this subsection is not written until the underlying data is filled in, so no placeholder numbers appear in this draft: system scale at the audited baseline; per-stage verification wall-clock against build time; defects caught before deployment by category, with confirmation status where tracked; and production incidents before/after gating, normalized per release, with confounders listed explicitly rather than a bare percentage drop.

5.2]

Prototype Measurements [P]

Table 3: Gate cost by ATM scale, mean ± std over 5 oracle seeds, layer = system.

Scale	Comp.	Analysis (s)	Detect (s)	Gate (s)
tiny	29	0.01 ± 0.00	0.00 ± 0.00	0.01 ± 0.00
small (`atm_system`)	74	0.05 ± 0.00	0.00 ± 0.00	0.05 ± 0.00
medium	148	0.13 ± 0.00	0.00 ± 0.00	0.13 ± 0.00
large	296	0.52 ± 0.02	0.01 ± 0.00	0.53 ± 0.02
xlarge	444	0.99 ± 0.03	0.02 ± 0.00	1.01 ± 0.03

Two studies. The **primary study** (§5.2.1–§5.2.2) evaluates a family of five systems in the deployment's own domain — air traffic management — at five scales, regenerating from a single committed artifact produced by an automated seed sweep. The **secondary study** (§5.2.3), carried forward unchanged from the prior revision, checks whether the primary study's pattern generalizes across *other* domains rather than across scale within one.

The primary study's five systems — 29, 74 (the original `atm_system` scenario, unchanged), 148, 296, and 444 components — are generated from configs that hold QoS mix, fan-out shape, and criticality proportion identical across scales and vary only entity counts, so **scale is the sole independent variable** — an improvement on the secondary study's corpus, where domain and scale changed together and could not be separated. Each of the five is evaluated at five oracle seeds ({42, 123, 456, 789, 2024}), 25 runs total, zero failures.

5.2.1 Verification Cost by ATM Scale. End-to-end cost of one gate invocation — graph analysis plus all detectors — against component count, domain held fixed, mean ± std over the five seeds:

Two observations. First, **detection is cheap here too, more so than the secondary corpus**: even at 444 components the full detector suite costs 0.02 s, and summed across all five scales the costliest single detector (cycle detection) averages 0.005 s — cheaper than the secondary corpus's already-cheap 0.047 s (§5.2.3), because this domain's fixed fan-out shape keeps the dependency graph sparse at every scale.

Second, **this controlled sweep resolves an ambiguity the secondary corpus left open.** There, analysis cost jumped sharply between 326 and 520 components (a 25× increase for a 1.6× size increase), and it was unclear whether that reflected component count or the specific topology of that one 520-component scenario. Here, holding domain and topology shape fixed and scaling components alone from 29 to 444 (a 15× increase) produces a **roughly proportional** cost increase — 0.01 s to 0.99 s, about 100× — with no cliff. The secondary corpus's steep jump was therefore a property of that one scenario's structure (or, per its own wall-clock caveat, of host load at measurement time), not a general law that cost explodes past a few hundred components. For this paper's motivating domain, at least, cost growth is gentle.

5.2.2 Detector Catalog Against a Cascade Oracle, by ATM Scale. We scored the catalog's flagged set against the oracle's critical set — boxplot top tier ($\geq Q_3$) once a scale has enough scored components to estimate quartiles, a top-20% mask below that, the same rule applied elsewhere in this codebase — at each of the five scales. Unlike §5.2.1's cost measurement, this result is a genuine trend across scale rather than a single robustness check, so we report one row per scale instead of pooling:

Table 4: Rule-based detection vs. simulated cascade impact, by ATM scale. "Seed-inv." scales report a single value (all 5 seeds bit-identical); xlarge — the one exception — reports mean ± std (4 of 5 seeds agree exactly; one seed's critical set differs by a single component).

Scale	Comp.	Prec.	Rec.	F_1	κ	%flag	S.I.
tiny	29	0.250	1.000	0.400	0.118	80.0%	Y
small	74	0.273	0.900	0.419	0.041	84.6%	Y
medium	148	0.269	0.900	0.414	0.031	85.9%	Y
large	296	0.250	0.949	0.396	0.000	94.9%	Y
xlarge	444	0.235±.002	0.868±.007	0.370±.003	−0.045±.005	93.2%	N

Agreement declines monotonically as the same ATM system grows, from $\kappa = 0.118$ (slight agreement) at 29 components to $\kappa \approx -0.045$ (no better than chance) at 444, crossing zero at the 296-component point. The mechanism is visible in the same table: the fraction of scored components the catalog flags climbs from 80.0% to 93–95% as scale increases, so precision erodes steadily (0.273 → 0.235) while recall stays high throughout (0.868–1.000). This is the same over-flagging mechanism identified in the secondary corpus (§5.2.3) — but here it is a genuine trend *within* one domain rather than a difference *between* domains: **as this ATM family grows, findings volume stays cheap to compute (§5.2.1), but the catalog gets steadily less discriminating.**

This is a mechanical consequence of how the catalog verdict is built, not evidence it fails to find critical components. Unlike the RM composite score $Q(v)$, threshold-matched to the oracle's critical-set size on both sides so precision/recall converge by construction, the catalog is a binary "flagged by at least one rule" decision whose flagged fraction is whatever the rule union produces, not a calibrated boundary: precision = base_rate × recall / flagged_fraction reproduces the measured precision to three decimals at every scale (xlarge: $0.252 \times 0.864/0.932 = 0.234$, measured 0.235), so **$F_1$'s trend is just the flagged-fraction trend read backwards**, not a failure to detect — recall never drops below 0.868. We read the *decline itself* more cautiously, since the generator recycles the same six roles at every scale rather than adding subsystems (§5.3).

5.2.3 External Validity: Generality Beyond ATM. The primary study asks whether verification quality holds as *one* domain scales. A separate question — does a pattern found on one domain generalize to *other* domains? — was the focus of the prior revision's evaluation, retained here as a secondary check. It used eight generated scenarios spanning six other domains (29 to 520 components, domain and scale varying together) plus three architectures transcribed from open-source systems (Autoware.universe ROS 2, the Train-Ticket microservices benchmark, and a cloud-native microservices mesh; 60–90 components), at the same five oracle seeds:

Agreement on the six other generated domains was near-chance ($\kappa \approx 0$), matching this paper's ATM finding at large scale, but agreement on the three *transcribed, real* architectures was materially better (fair, $\kappa \approx 0.30$), driven by less over-flagging (56.3% vs. 93.7–94.9%) — the same mechanical identity from §5.2.2 applies

Table 5: Rule-based detection vs. simulated cascade impact, cross-domain secondary corpus (unchanged from the prior revision).

Corpus	n	Prec.	Rec.	F_1	κ	%flag
Generated (6 domains)	8	0.237±.015	0.887±.063	0.374±.023	−0.036±.053	93.7%
Transcribed (real)	3	0.402±.073	0.865±.126	0.546±.077	0.299±.126	56.3%

here too. The pattern that best fits the evidence: **rule-based flagging is least discriminating on large, synthetically-generated graphs regardless of domain, and more discriminating on smaller and/or non-synthetic ones** — but scale and "generatedness" point the same direction here, so this corpus cannot cleanly separate which drives the effect. Caveats restated in §5.3.

5.3 Threats to Validity

- **Two systems, one name.** §5.1 and §5.2 measure different artifacts (§2.1). The prototype implements a subset of the deployed capability; prototype cost figures are lower bounds on deployed cost.
- **Simulated oracle.** §5.2.2 and §5.2.3 score detection against a simulator, not against observed field failures. The simulator is a model of cascade propagation, and agreement with it is not the same as agreement with reality. A second independent oracle in the same codebase agrees with this one at only $\rho \approx 0.40$, which bounds how far any figure measured on one transfers to the other.
- **Same six roles, recycled, not new subsystems.** The ATM generator's domain skin (§3.4) supplies six named roles and a fixed five-cluster hierarchy regardless of scale — growing from 29 to 444 components means more copies of the same six roles, not new subsystem types, which may not resemble how a real deployment adds capacity.
- **Effective sample size is 5 scale points, not 25 scale-seed pairs.** Four of five are seed-confirmed exactly, the fifth to within std ≤ 0.005: this rules out a lucky seed, but one generator draw per scale is one trajectory, not several independent ones.
- **Corpus composition.** The primary study is entirely single-domain and synthetic. The only cross-domain, partially non-synthetic evidence is the secondary corpus (§5.2.3): $n = 8/n = 3$, seed-invariant for 10 of 11 scenarios, a 2× wall-clock discrepancy against an earlier measurement, and hand-built (not harvested) transcriptions.
- **Confounders.** Gating was not the only change in §5.1's production-incident observation window; listed confounders bound the causal claim.

6 Related Work

Architecture description languages. AADL and SysML model hardware/software interaction and support analysis of the same properties we target [3], but the model is authored separately from the implementation and drifts. SaaG's model is reconstructed per candidate build from artifacts the build already produces, trading expressiveness for currency.

Static code analysis and runtime observability. SonarQube and Coverity analyze source and local control flow, structurally unable to see topic bindings, QoS contracts, or node topology since the system exists only in the composition of individually-analyzed units. Observability platforms (Prometheus, Dynatrace) see the deployed system with fidelity no static model matches, but only after a non-conforming artifact is already running. SaaG complements the former at the architectural boundary and, through the specified drift detection of §3.4, aims to route the latter's kind of observation back into pre-deployment verification.

Digital twins. The term is established in manufacturing and Industry 4.0, where a digital twin synchronizes a model against a physical asset via continuous sensor telemetry [8]. The object and fidelity are both different here: the twin models a *software architecture*, not a physical asset, and is *static* (§2.1), reconstructed per candidate build rather than continuously live. We use the term because the reconstruct-and-audit pattern is the same one digital-twin systems apply to physical assets, and the specified, not-yet-built dynamic half (§3.4) is exactly the telemetry-synchronization step the literature describes; we do not claim a live-synchronized model we have not built.

Middleware dependability. Pub/sub dependability has been studied at the protocol, broker overlay, and runtime levels [1, 2]. Our contribution is neither a protocol nor a runtime mechanism but a pipeline-integrated static check over the deployment's own descriptors.

Scalability testing. Scalability faults can stay latent until production scale [5, 11]. Existing techniques run the implementation: emulation [4, 7, 10], workload extrapolation [6], or scaling runs with static analysis [11]. SaaG runs nothing, so it needs no scaled deployment and sees no code-level fault.

7 Conclusion

We reported an architectural digital twin, a requirements baseline, an open prototype, and a CI/CD gate, with measurements from both an industrial deployment and the prototype (§1.2). The findings we would most want carried forward are the negative ones, plus one that surprised us. Building the graph and running structural rules is cheap regardless of scale (§5.2.1), but ranking findings by consequence gets worse as the same system grows: rule-based severity agrees with simulated cascade impact at $\kappa = 0.12$ (slight) on the smallest ATM system and $\kappa \approx -0.05$ (chance) on the largest — same domain, same generator template, only scale different (§5.2.2). A secondary check found the same near-chance pattern on other synthetic domains but materially better agreement on transcribed real architectures (§5.2.3); scale and synthetic generation both point toward the failure mode, and this paper's evidence cannot separate which is the true cause. Four of the seven specified checks we have not built are blocked not on analysis technique but on **getting configuration data into the model at all** (§3.4): budget for data acquisition, not graph algorithms.

Future work. Delta-aware gating with a waiver register (§4.1); reader/writer-granularity QoS conformance (§3.4); drift detection via the specified field-record path; and, highest priority, confirming the within-domain κ decline against a real, growing deployment rather than a synthetic scale sweep of one generator template.

References

[1] Antonio Carzaniga, David S. Rosenblum, and Alexander L. Wolf. 2001. Design and Evaluation of a Wide-Area Event Notification Service. *ACM Transactions on Computer Systems (TOCS)* 19, 3 (2001).

[2] Patrick Th. Eugster, Pascal A. Felber, Rachid Guerraoui, and Anne-Marie Kermarrec. 2003. The Many Faces of Publish/Subscribe. *Comput. Surveys* 35, 2 (2003).

[3] Peter H. Feiler and David P. Gluch. 2012. *Model-Based Engineering with AADL: An Introduction to the SAE Architecture Analysis & Design Language*. Addison-Wesley.

[4] Diwaker Gupta, Kashi V. Vishwanath, and Amin Vahdat. 2008. DieCast: Testing Distributed Systems with an Accurate Scale Model. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 407–421.

[5] Tanakorn Leesatapornwongsa, Cesar A. Stuardo, Riza O. Suminto, Huan Ke, Jeffrey F. Lukman, and Haryadi S. Gunawi. 2017. Scalability Bugs: When 100-Node Testing is Not Enough. In *Workshop on Hot Topics in Operating Systems (HotOS)*. ACM, 24–29. doi:10.1145/3102980.3102985

[6] Rong Shi, Yifan Gan, and Yang Wang. 2018. Evaluating Scalability Bottlenecks by Workload Extrapolation. In *IEEE International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*. IEEE, 333–347. doi:10.1109/MASCOTS.2018.00039

[7] Cesar A. Stuardo, Tanakorn Leesatapornwongsa, Riza O. Suminto, Huan Ke, Jeffrey F. Lukman, Wei-Chiu Chuang, Shan Lu, and Haryadi S. Gunawi. 2019. ScaleCheck: A Single-Machine Approach for Discovering Scalability Bugs in Large Distributed Systems. In *USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 359–373.

[8] Fei Tao, He Zhang, Ang Liu, and Andrew Y. C. Nee. 2019. Digital Twin in Industry: State-of-the-Art. *IEEE Transactions on Industrial Informatics* 15, 4 (2019), 2405–2415.

[9] TODO(author). 2026. Graph Neural Networks for Reliability and Dependability Analysis in Complex Distributed Systems based on Publish–Subscribe Architecture. (2026). Under review.

[10] Yang Wang, Manos Kapritsos, Lara Schmidt, Lorenzo Alvisi, and Mike Dahlin. 2014. Exalt: Empowering Researchers to Evaluate Large-Scale Storage Systems. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 129–141.

[11] Hao-Nan Zhu, Goodness Ayinmode, Cesar A. Stuardo, Haryadi S. Gunawi, and Cindy Rubio-González. 2026. Understanding and Detecting Scalability Faults in Large-Scale Distributed Systems. *IEEE Transactions on Software Engineering* 52, 10 (2026), 3166–3179. doi:10.1109/TSE.2026.3710096